Information Sciences 644 (2023) 119229

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Graph based encrypted malicious traffic detection with hybrid analysis of multi-view features

Yueping Hong [a], Qi Li [a,*], Yanqing Yang [a], Meng Shen [b]

[a] Beijing University of Posts and Telecommunications, No. 10 Xitucheng Road, Haidian District, Beijing, China
[b] Beijing Institute of Technology, No. 5 Yard, Zhong Guan Cun South Street, Haidian District, Beijing, China

ARTICLE INFO

Keywords:
Malicious traffic
Encrypted traffic
SSL/TLS
Multi-view features

ABSTRACT

At present, the TLS cryptographic protocol is widely deployed. While protecting the security and integrity of transmitted information, it also makes the detection of malicious behavior more difficult. In recent years, researchers have proposed many encrypted malicious traffic detection methods. However, the existing approaches have some shortcomings. Firstly, although researchers have extracted multi-view features from different aspects, which can be divided into vectorized features based on feature engineering and image features based on original data, existing methods cannot fully integrate the features of different forms of expression. Secondly, most of the existing methods do not fully analyze the correlation between different encrypted traffic. Thirdly, the existing methods based on correlation analysis have low processing efficiency and cannot be applied to real networks. In the paper, we present **MalDiscovery**, a novel technique to discover encrypted malicious traffic to address all the above issues. For encrypted malicious traffic, MalDiscovery constructs an attribute KNN graph, in which encrypted sessions are used as nodes to construct a KNN graph according to the similarity of image features, and vectorized features are used as attributes of corresponding nodes. After that, the GraphSAGE model is used to collect relevant node information through correlation analysis to enrich the embeddings of each node. Finally, we achieve the accurate binary classification of nodes in the graph based on richer embeddings. We use extensive experiments to evaluate the proposed method, and the experiment results show that MalDiscovery can achieve an accuracy of about 99.9%, significantly outperforming all compared methods.

1. Introduction

1.1. Background

To protect security during data transmission, cryptographic protocols have gradually developed and are currently becoming mature. According to the Google Transparency report [1], the proportion of encrypted traffic on the Internet has increased from 48% in 2014 to 99% in 2022. However, under the cover of cryptographic protocols, threat activities such as malware delivery, command-and-control channels, and data exfiltration, pose a serious threat to cyberspace security [2–4].

* Corresponding author.
E-mail address: liqi2001@bupt.edu.cn (Q. Li).

https://doi.org/10.1016/j.ins.2023.119229
Received 8 January 2023; Received in revised form 6 April 2023; Accepted 22 May 2023

Available online 1 June 2023

Among various cryptographic protocols, TLS protocol has been widely deployed in real networks since it can well protect the security and integrity of transmitted data and is the most representative protocol among cryptographic protocols. Besides, attackers are increasingly utilizing TLS-encrypted channels for the delivery and distribution of malware payloads, and for communication between infected hosts and command and control (C&C) servers. According to the MITRE ATT&CK report [5], many cyberattacks in the past few years have been related to port 443 of the TLS protocol. In addition, most of the existing public encrypted traffic datasets collected encrypted traffic data based on the TLS protocol, while paying little attention to other cryptographic protocols. Therefore, there is an urgent need to design an encrypted malicious traffic detection method for the TLS cryptographic protocols.

1.2. Shortcomings of existing approaches

As researchers increasingly focus on encrypted malicious traffic detection, many approaches have been proposed. Initially, traditional methods were mostly based on signatures. However, such methods cannot detect unseen malicious traffic while violating user privacy. Therefore, in recent years, researchers have chosen to use machine learning and deep learning methods to identify encrypted malicious traffic [6]. These approaches extract a variety of features from encrypted traffic and regard encrypted traffic as sample nodes, then use machine learning or deep learning models to detect the maliciousness of each node respectively according to the features of encrypted traffic itself. However, these methods have two drawbacks. First, existing methods extract a variety of encrypted traffic features from different aspects. These features can often be referred to as multi-view features by researchers, and they are divided into two categories: vectorized features and image features according to the representation form. Many researchers believe that multi-view features extracted from different aspects can complement each other and assist sample classification from different perspectives under appropriate feature fusion methods [7,8]. Nevertheless, existing studies usually use only one type of feature that ignore the supplementary information or directly concatenate several features that can destroy the correlation information of the image and introduce noise to vectorized features. Second, with the development of adversarial technology, attackers can easily interfere with the effect of the models by modifying features and other methods [9]. Fortunately, malicious threat activities are usually realized by multiple infected hosts jointly initiating network communication, and there must be rich correlations between these encrypted malicious sessions. Using this correlation information can help us more accurately detect encrypted malicious traffic. Nevertheless, most of the existing methods do not consider the analysis of the correlation between encrypted traffic. To fully analyze the correlation between encrypted traffic, researchers introduced graph-based models and proposed many methods. In these studies, encrypted traffic samples are constructed into a graph based on the correlation between different samples, and correlation-analysis models such as RandomWalk [10], GCN [11], etc. are utilized to discover the malicious sample nodes in the graph, so that realize the malicious detection of samples. However, these methods are inefficient in the phase of testing unknown traffic since they use transductive graph-based models. Therefore, it is an urgent problem to design a method that can achieve appropriate feature fusion, correlation analysis, and high efficiency.

1.3. Solution

Considering the detection of encrypted malicious traffic, three problems need to be solved. (1) How to effectively utilize the multi-view features of encrypted traffic samples to learn a unified embedding? (2) How to consider the correlation information of encrypted traffic samples in the testing phase? (3) How to improve detection efficiency? To address these issues, we make the following contributions:

- To fuse multi-view features skillfully, we construct an attributed KNN graph, in which we analyze the correlation between samples through the similarity of image features and combine the vectorized features as the attribute of each node in the KNN graph.
- To exploit the correlation for detection, we propose **MalDiscovery**, which utilizes a GNN model which can capture the correlation information between different nodes in the attributed KNN graph.
- To improve detection efficiency, we abandon the transductive models such as GCN [11] and utilize the inductive model GraphSAGE [12] for correlation analysis.

Our approach not only realizes the fusion of multi-view features and the analysis of the correlation between encrypted traffic, but also has high processing efficiency, which can be deployed in the real network environment.

2. Related work

Malicious traffic detection is important for establishing a secure and reliable computer network architecture. With the development of encryption technology, attackers also use it to evade the detection against their malicious activities, which brings a great challenge to intrusion detection systems [13]. Presently, researchers pay more attention to detecting encrypted malicious traffic. The research on encrypted malicious traffic detection has gone through a long period of development, which can be roughly divided into three stages: signature-based studies, ML-based & DL-based studies, and correlation-based studies.

Signature-based studies are based on the matches between the identification signatures and the actual information in encrypted traffic payloads. The signatures include the universal characteristics of an application protocol and are built via two sources: from

the standard protocol specifications and documentation, or manual observation and analysis. Deep Packet Inspection (DPI) [14–16] is the most well-known method for this type of research. These approaches are essentially a man-in-the-middle attack [17], which intercept encrypted traffic during transmission, detect malicious activities by using DPI through decrypted ciphertext information, then re-encrypt and forward traffic. However, these approaches destroy the secure channel in the procedure of decryption, which seriously violates the user's privacy. In addition, decryption and re-encryption are computationally expensive. In order to detect encrypted malicious traffic without decryption, some researchers have tried to extract information from outside the payload and use traditional machine learning (ML) or deep learning (DL) to analyze encrypted traffic samples.

Machine Learning-based (ML-based) & Deep Learning (DL-based) studies have gradually matured in the last decade, and these methods input the extracted encrypted traffic features into the models to determine the maliciousness of the samples. The extracted features can be divided into vectorized features and image features according to their representation form. ML-based methods rely on a large amount of prior knowledge to extract features and train shallow machine learning models on this basis to make full use of the observable features introduced by encryption mechanisms to distinguish encrypted malicious traffic. In this process, the features extracted by researchers through feature engineering are usually represented in the form of vectors, so these features are usually referred to as vectorized features in subsequent works. Anderson et al. [2–4] have long focused on encrypted malicious traffic detection and defined the detection granularity as a single encrypted session. Their works mainly focused on two statistical features of encrypted traffic: handshake information (version, cipher suite, extension, certificate, etc.) and session metadata (packet length sequence, packet time interval sequence, etc.). Wang et al. [18] extracted the features from HTTPS traffic and then fed them into shallow machine-learning models to detect encrypted malicious traffic. Shen et al. [19] proposed an attribute-aware encrypted traffic classification method based on the second-order Markov Chains to deal with the problem that the existing encrypted traffic classification methods have low classification accuracy for traffic with similar fingerprints. Dai et al. [20] proposed an encrypted malicious traffic detection method using multi-view features for SSL protocol. They extracted features from multiple aspects, including flow statistics, SSL handshake fields, and certificates, to retain the original key information, and used four machine learning models as classifiers to achieve the highest accuracy in the XGBoost model. Liu et al. [21] proposed a detection structure called MalDect for encrypted malicious traffic and uses the method of online random forest to detect the traffic before illegal operation. Shen et al. [22] proposed FineWP, a novel fine-grained webpage fingerprinting method, which extracts length information of packets and feeds into machine learning models for classification. Dong et al. [23] extracted packet length sequences and constructed MLTree for each encrypted session to analyze the malicious traffic behavior. However, the detection accuracy of these approaches greatly depends on prior knowledge. If the traffic packet changes or is confused by attackers, the effect of this method will be greatly reduced [24]. DL-based studies get rid of the limitation of prior knowledge [25] by converting the original encrypted traffic data into images and then automatically discovering malicious encrypted traffic with deep learning approaches. The images extracted during this process are called the image features of encrypted traffic by researchers, which can completely contain the information of the encrypted traffic. Wang et al. [26] proposed an end-to-end encrypted traffic classification method with one-dimensional convolution neural networks, which is the first time to apply an end-to-end method for the encrypted traffic classification domain. Shapira et al. [27] proposed an approach to transforming encrypted traffic into an intuitive picture and then using convolutional neural networks (CNN) to identify the traffic category. Lin et al. [28] converted packets into images and used CNN to generate corresponding embeddings, and then fed each image into a bidirectional LSTM in time order to obtain a classification of a flow. Bazuhair et al. [29] proposed an image-based encrypted traffic embedding method based on Berlin noise, which uses Perlin noise to encode the given connection features into the image to train the deep learning model for binary classification of connection flows. Wang et al. [30] proposed an encrypted traffic application identification method using a stack type of automatic encoder. Shao et al. [31] proposed a traffic data encoder based on deep learning to extract traffic representation and classify the traffic. However, ML-based and DL-based methods have certain drawbacks. Firstly, since in the field of encrypted traffic where the payload is not visible, the features that can be extracted and exploited are very limited, the analysis from the perspective of a single encrypted traffic feature cannot accurately detect malicious traffic. Secondly, with the development of adversarial technology, attackers have been able to modify the key features and disguise malicious traffic as normal traffic to evade detection.

Correlation-based studies are emerging ideas in the current encrypted malicious traffic detection research. With the increasing attention to correlation in the field of encrypted traffic research in recent years, researchers have tried to construct encrypted traffic data into a graph and use the models related to graph research to detect encrypted malicious traffic. Shen et al. [32] proposed a traffic interaction graph structure and utilized the graph classification method to identify distributed applications based on encrypted traffic. Wang et al. [33] presented a graph method on botnet analysis to show that the correlation between traffic is crucial information that must be carefully analyzed in malicious traffic detection. Cui et al. [34] proposed an approach that uses Siamese Heterogeneous Graph Attention Network to measure whether two IPv6 client addresses belong to the same user even if the user's traffic is protected by TLS encryption. Pham T D et al. [35] focused on Android encrypted traffic and designed a communication graph and utilized the graph model to realize the identifying mobile apps based on network traffic. The nodes in the communication graph are defined by tuples of IP addresses and ports of the services connected by the app, and edges are established by the weighted communication correlation among the nodes. However, these methods currently do not consider the fusion of traffic features, and most of the graph models they used are transductive, which does not have good generalization.

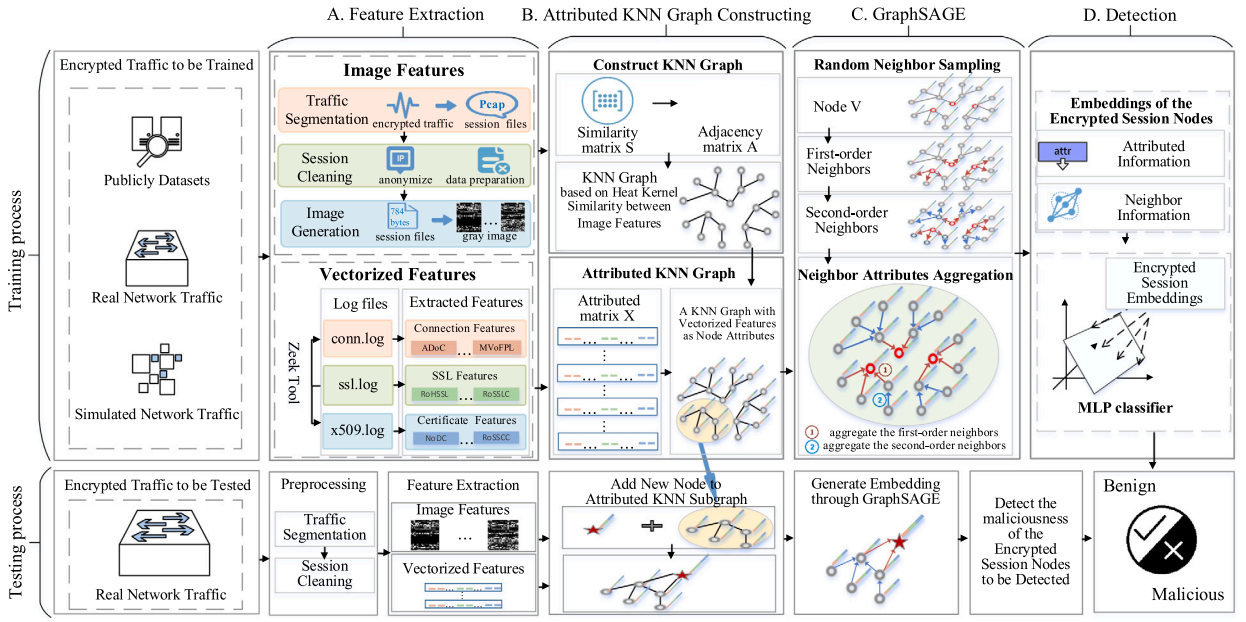


Fig. 1. Overview of MalDiscovery.

3. Methodology

3.1. Overview

Fig. 1 shows the overall framework of MalDiscovery. We can be divided the proposed approach into two relatively independent links: offline training and online testing. The reason we designed MalDiscovery like this is to prevent the slower training phase in the model from affecting the detection efficiency of the model. An excellent model structure should be able to save time and space complexity in the running of the program. Our method has the advantage of high testing efficiency, which is derived from the inductive model GraphSAGE we use. During the offline training phase, we can collect a large amount of training data, which allows the system to use a lot of time to complete the training of the model. In the online testing phase, different from other models, we will complete the efficient processing of samples to meet the efficiency requirements of online traffic detection.

From another view, our approach is mainly consisting of four modules: feature extraction module, attributed KNN graph construction module, GraphSAGE module, and detection module. In the feature extraction module, we extract the image features and vectorized features of encrypted traffic; in the attributed KNN graph construction module, to fully integrated these two types of features, we construct the attributed KNN graph structure of encrypted traffic based on graph modeling method, in which the similarity of image features is used to construct an undirected KNN graph, and the vectorized features are used as the attributes of the nodes in the graph; in GraphSAGE module, due to the relevant sample features extracted through the correlation information between nodes have significant reference value for the malicious detection task of the target node, we explore the correlation information of encrypted traffic samples through random neighbor sampling and neighbor attributes aggregation to obtain rich node embeddings of encrypted traffic. In this procedure, the GraphSAGE model shows a more appropriate computing cost and time cost than GCN. Since node embeddings are generated by GraphSAGE analysis of correlations and aggregated relevant information, they can more comprehensively portray the encrypted sessions, and then make our approach achieves more accurate maliciousness detection. Finally, in the detection module, we feed the embeddings of nodes into a classifier to perform binary classification of the nodes in the attribute KNN graph by an MLP classifier. Since the nodes in the attributed KNN graph correspond to encrypted sessions, the binary classification of the nodes in the graph by MLP is equivalent to the binary classification of whether the encrypted session is malicious. In addition, if researchers want to migrate our method to other cryptographic protocol scenarios, they only need to replace the feature extraction module with the feature extraction module based on other cryptographic protocols. It means that our model is not limited to the analysis of TLS-encrypted traffic.

In this paper, we implement the detection task of encrypted malicious traffic at the session level. On the one hand, the bidirectional flow solves the problem of distinguishing between client and server. On the other hand, the session contains more abundant traffic information and more detailed labels. After defining the session as the granularity of the encrypted malicious traffic detection task, we need to complete the feature extraction of the encrypted sessions.

3.2. Feature extraction

In the existing works on encrypted traffic, researchers have extracted a variety of features of encrypted traffic from different aspects. These features are often presented in two forms, namely, vectorized features and image features. The vectorized features

Table 1
Extracted vectorized features.

Feature	Log	Description
ADoC	conn	Average Duration of the Connection
SDoD	conn	Standard Deviation of Duration
MIToC	conn	Minimum Inter-arrival Time of Connection
AIToC	conn	Average Inter-arrival Time of Connection
MIToC	conn	Maximum Inter-arrival Time of Connection
NoBiU	conn	Number of Bytes in Upstream
NoBiD	conn	Number of Bytes in Downstream
RoDBaAB	conn	The ratio of Downstream Bytes
IPN	conn	Inbound Packets Number
OPN	conn	Outbound Packets Number
RoIaOB	conn	The ratio of Inbound to Outbound Bytes
MVoFPL	conn	Mean Value of the Forward Packet Length
MVoBPL	conn	Mean Value of Backward Packet Length
RoHSSL	SSL	The ratio of SSL connections that use the high version-TLS
APKL	SSL	Average Public Key Length
RoSNI	SSL	The Ratio of SSL connections that has SNI (Server Name Indication) extension
ISNIIP	SSL	Indicates if SNI IP is the same as destination IP
RoSSLC	SSL	The ratio of SSL connections that has certificate
NoDC	Cert	Number of Different Certificates
ACL	Cert	Average Certificate Length
AAoC	Cert	Average Age of Certificates
AoCV	Cert	Average of Certificate Validity
SDoCV	Cert	The standard deviation of certificate validity
RoSSCC	Cert	The ratio of SSL connections that use Self-Signed-Certificate

take the encrypted traffic information extracted based on feature engineering as the value in the vector. The image features are to process all the information in the traffic into binary form, and each byte is corresponding to a grayscale image pixel. These two categories of features can be said to cover all the features in encrypted traffic research. Even if new encrypted traffic feature extraction methods are proposed, the extracted features can still belong to one of these two categories.

Vectorized Feature

The Vectorized features are extracted based on feature engineering, including handshake information and session metadata. The handshake information includes the communication configuration, such as the version, and extension, which usually varies from site to site. Session metadata contains packet length sequence, packet time interval sequence, and other information, which can reflect the communication behavior of the encrypted session. Since the research on this category features [36] is very mature and there are many extraction tools for vectorized features, we directly use the tool Zeek [37] to analyze encrypted traffic samples and generate log files for each session including conn.log, ssl.log, and x509.log. When processing each connection, the tool will allocate a unique index for each session and generates the corresponding log files. The relationships between log files can be connected by the unique index. The conn.log file contains information about the connection, such as source IP address, destination IP address, port, connection duration, and number and size of upstream and downstream packets. The ssl.log file contains information about SSL / TLS, such as timestamp, version, key, server name, etc. The x509.log file contains information about the connection certificate, such as certificate serial number, version, issuer, validity period, server DNS, the type of key, and the length of the key, etc. Therefore, the extracted features can be divided into three categories: connection features, SSL/TLS features, and certificate features. These three features complement each other and retain enough original information, to represent session-level encrypted traffic. The vectorized features extracted in our paper are shown in Table 1.

Image Feature

Since the image features are derived from the raw data of encrypted traffic, they can retain the original content of traffic to the greatest extent. During TLS communications, plaintext data of handshake is exchanged between the client and server to negotiate the parameters used in subsequent encrypted connections, such as the version, cipher suite, extensions, and certificate. Previous studies have demonstrated that these pieces of information are effective in encrypted malicious traffic detection. Therefore, image features have received much attention and utilization in encrypted malicious traffic detection research. However, in encrypted traffic, the image generated by the application data after the handshake has a lot of noise, so the selection of image size M is particularly important in the process of extracting image features. If the selected content is too small, the image will not fully describe the sample information, and if the selected content is too large, the information redundancy will affect the accuracy and efficiency of the detection model. To completely retain the handshake information and appropriate application data, we select the first 784 bytes of the original encrypted session for gray image generation after sufficient experiments, which are divided into three important stages: traffic segmentation, session cleaning, and image generation.

a) traffic segmentation
   In this process, we use the session as the granularity of segmentation, and the continuous original encrypted traffic is divided into multiple session files, which are saved as pcap files.
b) session cleaning

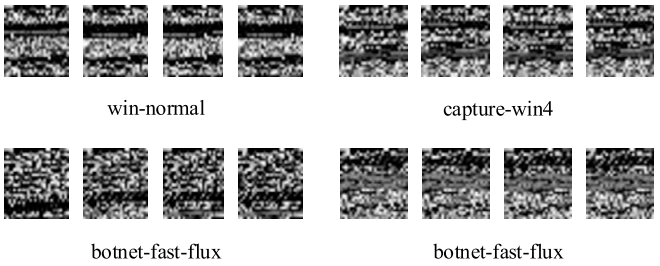


Fig. 2. Image Patterns for different types of applications.

We anonymized the MAC address and IP address in the data link layer and network layer to remove the specific messages that interfere with the classification results; and then, we deleted empty flow and duplicate data that may lead to misjudgment of the model.

c) image generation

Firstly, we deal with the cleaned session files with uniform lengths in the same size, which is 784 bytes. When the length of the file is greater than 784 bytes, it will be intercepted. If it is less than 784 bytes, it will be supplemented with 0x00 at the end of the file. Since each byte can be converted into an integer in the range of [0, 255], each byte can correspond to a gray pixel value. After that, we reshaped the byte sequence into a matrix of 28×28 pixels, and further construct it into a gray image with the size of 28×28.

3.3. Attributed KNN graph construction

For fusing different types of features, the most direct method is to use different individual learning models and use heterogeneous integrators. However, it has the following disadvantages: (1) each type of feature is learned independently, which cannot get more abundant embeddings of encrypted traffic; (2) it ignores the relationship between encrypted traffic sample data; (3) heterogeneous integrator method based on different types of individual classifiers is time-consuming. To solve these problems, we designed an attributed KNN graph construction module, which can fuse the image features and vectorized features at the session level, and integrally describe the correlation information between encrypted sessions.

Image features contain complete encrypted session information, including encrypted session handshake information and application data. The handshake information can reflect the specific configuration of the encrypted session, and these configurations are often reused in the encrypted sessions participated by the same server. And application data can reflect the type of network application to a certain extent. As shown in Fig. 2, since different malware families and applications have different image patterns, we construct KNN graphs based on the similarity between image features.

The specific process of converting the encrypted traffic into a KNN graph with node attributes is shown in part B of Fig. 1, where the KNN graph can be understood as a form of a similarity matrix S. By measuring the distance between any sessions through image features, the similarity between the closer sessions is higher, and the similarity between the farther sessions is lower, even negligible. The conversion process is as follows:

According to image features, each session of encrypted traffic is regarded as a node in high-dimensional space, and Heat Kernel is used to calculate the similarity between image features of different encrypted sessions to construct a similarity matrix S.

The similarity between session samples i and j calculated by Heat Kernel is shown as eq. (1).

$$S_{i,j} = e^{-\frac{||x_i - x_j||^2}{t}} \tag{1}$$

where t is the time parameter in the heat conduction equation.

According to the similarity matrix S, the first K similar session samples of each sample are selected as its neighbor nodes to construct an undirected KNN graph. In this way, we can abstract the graph structure of encrypted traffic from non-graph data, which can be represented by adjacency matrix $\mathcal{A}$.

Then, an $N \times d$ attributed matrix $\mathcal{X}$ is constructed by using vectorized features, where N is the number of encrypted sessions, d is the dimension of each vectorized feature, and i-th row of $\mathcal{X}$ is the attribute and initial embedding vector of the i-th session node in KNN graph.

So far, we have successfully constructed the undirected attributed graph by using the image features and vectorized features of encrypted traffic, and $\mathcal{G}$ can be defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ is the node set; $\mathcal{E} = \{e_{ij}, e_{pq}, e_{xy}, \dots\}$ is the undirected edge set; $\mathcal{X} = \{x_1, x_2, \dots, x_N\}$ is the node attribute set. Graph $\mathcal{G}$ skillfully integrates the vectorized and gray image features of encrypted traffic, and clearly represents the relationship between encrypted traffic nodes.

3.4. GraphSAGE

The basic idea of GCN is to aggregate the attributes of neighbor nodes to generate the embeddings of target nodes. Inspired by this idea, GraphSAGE is applied to encrypted malicious traffic detection. However, different from GCN, which belongs to the transductive

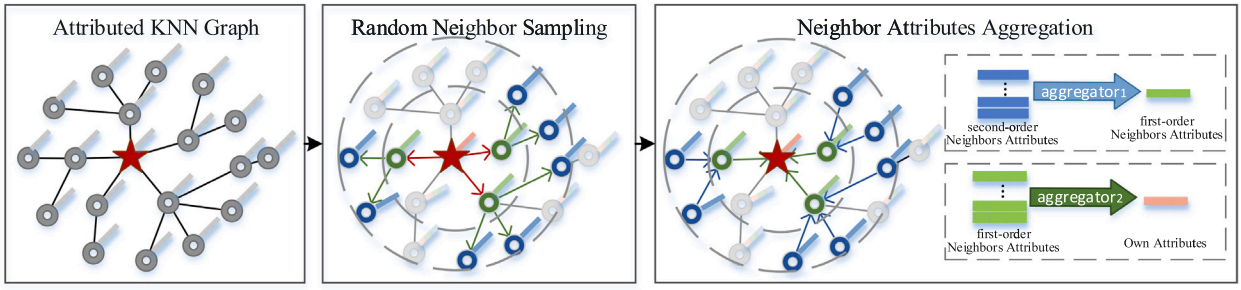


Fig. 3. Process of GraphSAGE.

model, GraphSAGE, which belongs to the inductive model, has better generalization ability. GCN must use the same graph during training and testing, in other words, GCN can only detect nodes that already exist in the graph at the time of training, and needs to be retrained when encountering nodes that have not been seen. While GraphSAGE can utilize its node sampling strategy during both the training and testing phases to generate subgraphs that are used to aggregate node representations. Therefore, when GraphSAGE encounters nodes that have not been seen during testing, it only needs to generate a subgraph to calculate the node representation without retraining the model, thus achieving higher detection efficiency than GCN. The GraphSAGE model mainly contains two key strategies: random neighbor sampling and neighbor attribute aggregation (see Fig. 3).

On the one hand, the random neighbor sampling strategy can capture the low-order and high-order correlation information of encrypted sessions. The low-order correlation information refers to the direct relationship between encrypted sessions, while the high-order correlation refers to the multi-hop relationship that imposes similarity constraints on data samples. Taking the second-order correlation as an example, for two nodes without a direct relationship, if they have a common neighbor, they should still have a similar embedding. Therefore, it is not enough to extract the low-order correlation information of encrypted sessions, and the capture of high-order relations is also important.

On the other hand, in the neighbor attributes aggregation mechanism, it is worth noting that, we train an aggregation function, which aggregates information from different hops to generate the embedding of the target node, instead of directly training a different embedding vector for each node. Then, the embeddings of nodes can be used for downstream classification tasks.

Here, we describe how to obtain the embedding h_v^l of encrypted session node v in layer l based on random neighbor sampling and neighbor attributes aggregation mechanism. Taking the second-order neighbor as an example, we first obtain the first-order neighbor set $N_{(v)}^1$ and the second-order neighbor set $N_{(v)}^2$ of node v. Specifically, the hyperparameter S_l is used as the sampling rate of each node in the l-layer, that is, the l-layer sampling neighbors of each node cannot exceed S_l.

After that, the neighbor attributes aggregation mechanism is performed layer by layer. We aggregate the embeddings of the second-order neighbors to get the embeddings of the first-order neighbors and then aggregate the embeddings of the first-order neighbors to get the embedding of node v. Take the l-layer embedding of node v obtained by aggregating the neighbors of layer $l = 1$ as an example. The specific aggregating operations are as follows:

Firstly, for each node $v \in \mathcal{V}$, the features of its neighborhood nodes $N_{(v)}$ are integrated by aggregation operation, which is expressed as follows:

$$h_{N_{(v)}}^l \leftarrow Agg(\{h_{u_i}^{l-1}, \forall u_i \in N_{(v)}\}) \tag{2}$$

The features of the upper layer of the central node $h_{u_i}^{l-1}$ are concatenated with the aggregated neighborhood features $h_{N_{(v)}}^l$. Then the concatenated vector is input into the single-layer network with a nonlinear activation function to obtain the l-layer embedding of the central node v. The calculation process is as follows:

$$h_v^l \leftarrow \sigma(W^l \cdot CONCAT(h_v^{l-1}, h_{N(v)}^l)) \tag{3}$$

where W^l is the weight matrix of l-layer, and σ is the Sigmoid function.

After regularizing the embeddings of nodes, it can be used in the next iteration.

$$h_v^l \leftarrow h_v^l / ||h_v^l||_2, \forall v \in \mathcal{V} \tag{4}$$

Once getting the last embeddings of nodes, we input it into the supervised classifier of detection module to identify the encrypted malicious traffic.

3.5. Optimization objectives

Through the above algorithm, we can aggregate multi-hop neighbor information through the message-passing mechanism based on the correlation between nodes, so as to obtain the embedding of encrypted session nodes with a certain aggregation depth, which contains the attribute information of the target node and its low-order and high-order neighbor information. Then, we feed the embeddings of encrypted session nodes into the MLP classifier and softmax function to perform binary classification on the

corresponding encrypted session nodes. Since the nodes in the attribute KNN graph directly correspond to the encrypted session data, the binary classification of the nodes in the graph is equivalent to the binary classification of the encrypted session. When the classification result is 1, it means that the encrypted session is malicious, and if it is 0, then indicates that it is a benign encrypted session.

$$Z_C = softmax(mlp(h_v^l)) \tag{5}$$

As shown in the eq. (6), we choose the cross-entropy loss function to guide the optimization of the model, so that the model can learn a better aggregation function and then aggregate the neighbor information to get better embeddings of nodes.

$$Loss = -\frac{1}{N}\sum_{n=1}^{N}\sum_{C=0}^{1} y_C log(Z_C) \tag{6}$$

Where N is the total number of encrypted sessions, y is the label, $C = 0$ is benign encrypted sessions, and $C = 1$ is encrypted malicious sessions.

4. Experiment

4.1. Datasets

We extracted encrypted traffic samples from two experimental datasets, CTU-13 and MCFP, and evaluated the detection performance of the proposed method. We published a portion of our code and the dataset extracted from CTU-13 and MCFP to Github.[1]

CTU-13 [38] is a dataset of traffic generated by specific malware captured in 13 different scenarios at the Czech Technical University (CTU) in Prague. In each scenario, CTU executed a specific piece of malware that uses multiple protocols and is capable of multiple different operations to generate malicious traffic. This malware includes Dridex-A, Kazy, Upatre, Zbot, Neris, Rbot, etc., all of which can be detected by multiple detection engines on VirusTotal [39]. Whereas normal traffic comes from data transfers, social network communications, and the use of browsers. As early as 2008, botnet attacks began to use traffic cryptographic protocol to hide the maliciousness of network behavior on a large scale to evade traditional security protection and detection. Therefore, in 2011, a certain amount of encrypted traffic data was collected in the CTU-13 dataset. We extracted 2619 malicious encrypted sessions from the CTU-13 dataset.

MCFP [40] dataset was released by the Czech Technical University in Prague (CTU) between 2013 and 2018 in its Malware Capture Facility Project to capture and analyze persistent malware traffic. It serves a large amount of botnet attack traffic and benign traffic that communicates using TLS protocol. In the early years, this dataset only contained a small amount of encrypted traffic, but as cryptographic protocols were widely deployed, it is more and more inclined to collect encrypted traffic in the update of the MCFP dataset. In recent years, the proportion of encrypted traffic in MCFP has gradually increased, which contains far more encrypted traffic than CTU-13. During the experiment, we only kept the complete encrypted session for analysis and extracted 6 malicious pcap-files from CTU-Malware-Capture-Botnet-153-1, CTU-Malware-Capture-Botnet-173-1, CTU-Malware-Capture-Botnet-240-1, CTU-Malware-Capture-Botnet-241-1, CTU-Malware-Capture-Botnet-275-1, CTU-Malware-Capture-Botnet-322-1 in the MCFP dataset, which contained 61,101 malicious encrypted sessions. For benign samples, we extract 13 pcap-files from CTU-Normal-20 to CTU-Normal-32 of MCFP, including 69358 benign encrypted sessions.

After collecting the encrypted traffic data, we process the data extracted from the CTU-13 and MCFP datasets at the session level and label the corresponding encrypted session in the pcap-file according to the label of the pcap-file in the dataset to obtain the explicit label of the encrypted sessions. In our experiments, we label encrypted malicious traffic as 1 and normal traffic as 0. Since background traffic involves user privacy, it is not used as an experimental dataset in this paper. As a result, we obtained 63720 encrypted malicious traffic and 69358 encrypted benign traffic as the dataset for this experiment, 80% of which are used as the training dataset, 10% are used as the verification dataset, and 10% are used as the test dataset. The specific data distribution of malicious encrypted traffic is shown in Table 2.

4.2. Baselines

We selected several common machine learning (ML) and deep learning (DL) models in the field of encrypted traffic research and three state-of-the-art models published in recent years as baselines. (1) SVM [41]: it is a typical shallow learning method based on vectorized features and is often used to detect malicious traffic. (2) Linear Regression (LR) [4]: a generalized linear regression analysis model. (3) Decision Tree (DT) [42]: a basic classification method based on a tree structure, and its input is usually vectorized features. (4) Random Forest (RF) [43]: an integrated classifier containing multiple decision trees, which is very flexible and has low computational overhead. (5) AdaBoost [44]: an iterative algorithm whose core idea is to combine several weak classifiers to form a stronger final classifier. (6) CNN: it is one of the representative algorithms of deep learning and is widely used in the field of image processing. Shapira et al. [27] converted traffic into images and utilized CNN for encrypted traffic classification. (7) TSCRNN: It is a method proposed by Lin et al. [28] It uses CNN to generate the embeddings of the images corresponding to packets, and then uses

[1] http://github.com/NatsuHB/MalDiscovery.

Table 2
The composition of malicious encrypted traffic samples.

Dataset	Pcap	Encrypted sessions	TCP sessions
CTU-13	CTU-Malware-Capture-Botnet-42	137	12469
	CTU-Malware-Capture-Botnet-43	81	21495
	CTU-Malware-Capture-Botnet-44	2227	32151
	CTU-Malware-Capture-Botnet-46	141	852
	CTU-Malware-Capture-Botnet-47	33	4533
MCFP	CTU-Malware-Capture-Botnet-153-1	2504	3095
	CTU-Malware-Capture-Botnet-173-1	18478	20048
	CTU-Malware-Capture-Botnet-240-1	11085	11233
	CTU-Malware-Capture-Botnet-241-1	8752	8943
	CTU-Malware-Capture-Botnet-275-1	8226	8683
	CTU-Malware-Capture-Botnet-322-1	12056	16086

bi-directional LSTM to process multiple embeddings to realize the classification of the flow. (8) DeepWalk: is a basic algorithm for network embedding learning and a classification method based on graph structure, which was mentioned in the work of Fu et al. [10] (9) GCN: is a classic GNN model. Pham T D et al. [35] constructed a communication graph and utilized GCN to realize encrypted traffic classification (10) MalDiscovery: the proposed method.

Among them, machine learning-based (ML-based) methods, such as SVM, LR, DT, RF, and Adaboost, are commonly utilized in existing works. CNN and TSCRNN are deep learning-based (DL-based) methods proposed in recent years. And the DeepWalk and GCN are often utilized for correlation analysis in the field of encrypted traffic.

Different baseline methods have different requirements on the form of their input data. To comprehensively evaluate the effect of the proposed attribute KNN graph in feature fusion while satisfying the input requirements of different methods, we design the input into five forms. Among them, F1 represents the vectorized features of encrypted traffic, F2 represents the simple concatenating of vectorized features and image features, F3 represents the image features, F4 represents the correlation information of sample data, and F5 represents the attributed KNN graph of encrypted traffic datasets. The “concatenating” here means to encode the image features in bytes and flatten them into a one-dimensional numerical vectors with each element value between 0-255. And the “correlation information” in F4 means to keep only the graph structure constructed based on correlation between sessions, where nodes correspond to encrypted sessions and nodes do not have attributes.

4.3. Parameter and metrics

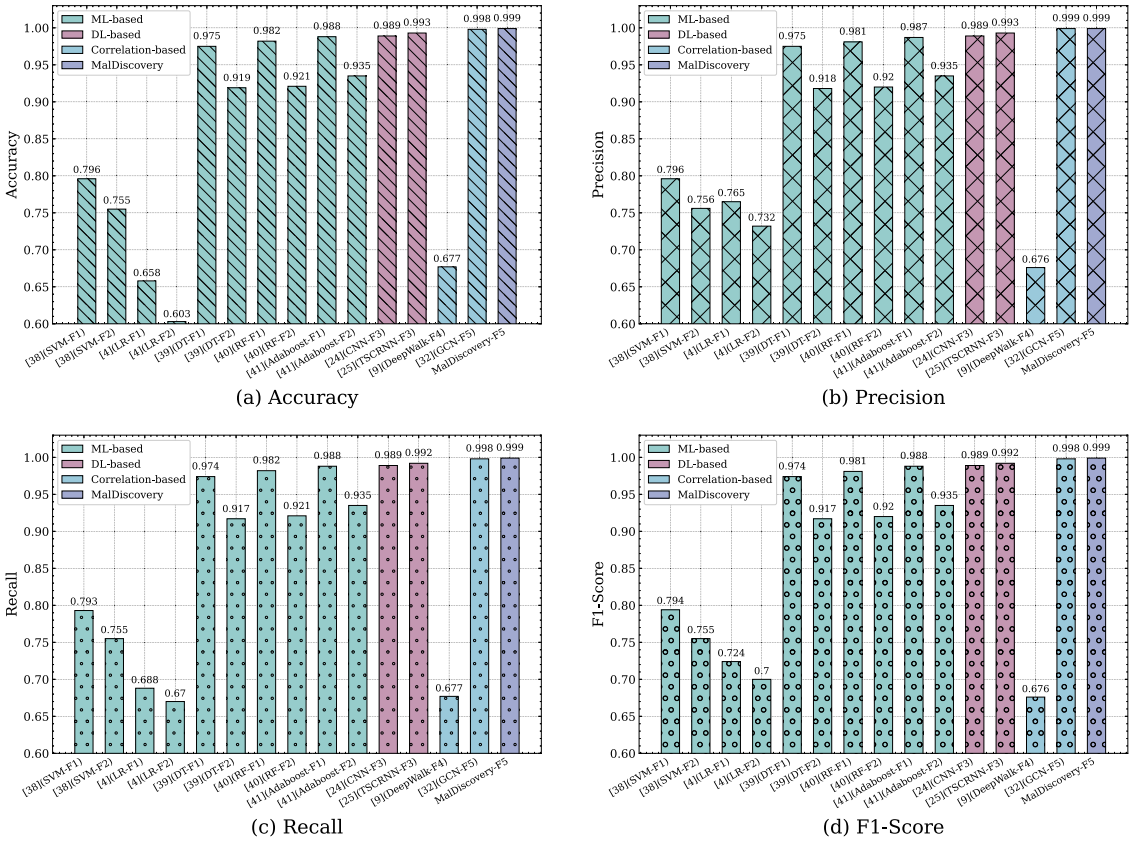
For baselines, we set them as the default parameter of the model. For MalDiscovery, the dimension of the node embeddings we obtain through GraphSAGE is set to 100, and the learning rate to 0.001. For the convenience of experimental observation, we set that the sampling number S_l of each layer is equal, and use S to refer to it. We also select the best-performed hyper-parameters $S = 10$, $K = 15$, and MeanPooling aggregation function on the validation set. In addition, we adopt ACC, Precision, Recall, and macro-F1 as evaluation indexes.

4.4. Experiment result

Fig. 4 shows ACC, precision, recall, and macro-F1 of different methods we use to detect encrypted malicious traffic. Note that MalDiscovery is significantly better than the other baselines in almost all four indicators, indicating the effectiveness of our proposed model.

For the baseline method, we have the following observations. Firstly, the performance of end-to-end detection based on image features is better than that of shallow machine learning detection based on vectorized features. The reason is that with the introduction of encryption technology, the features of feeding shallow machine learning models are artificially designed based on encryption protocol, which has the problem of missing information. The end-to-end model based on image features makes full use of the original information of encrypted traffic. Secondly, the performance of shallow machine learning methods that use new features directly concatenated from two categories of features is far inferior to the shallow machine learning method using only vectorized features. The reason is that the method flattens the image features directly, which destroys the correlation information of the grayscale image and introduces the noise from the image, which affects the effect of the shallow machine learning model. At the same time, the method does not consider the relationship between samples. In addition, we can observe that the detection performance of the DeepWalk method is the worst because DeepWalk only considers the relationship between encrypted traffic data samples and ignores the attribute features of the data itself. The TSCRNN performs better than the simple CNN in the experiments. The reason is that TSCRNN inputs the embeddings into RNN according to the time order of the packets after using CNN to generate embeddings for each packet according to the image features. This method can take into account the temporal correlation between encrypted traffic. Finally, the Mappgraph method fully analyzes the rich correlation information contained in the attributed KNN graph by using GCN and achieves a performance second only to MalDiscovery.

After that, by comparing the results of MalDiscovery and other methods, we believe that the proposed encrypted malicious traffic detection approach based on GraphSAGE is an effective method to fuse the image features and vector features of encrypted traffic.



(a) Accuracy (b) Precision

(c) Recall (d) F1-Score

Fig. 4. Performance of Different Methods.

Firstly, Fig. 4 shows that the performance of MalDiscovery is significantly better than that of the method using only one type of feature, which means that the fusion of multi-view features is beneficial to improve the performance of encrypted malicious traffic detection. Secondly, the performance of concatenating directly method is not improved or even declined. This kind of mixing operation may introduce noise, which greatly affects the judgment of the model. Therefore, it is necessary to design an end-to-end encrypted malicious traffic detection model integrating various types of features.

In addition, we use different parameters to train our approach and show the training results in Fig. 5.

4.5. Aggregator selection

We compared four variants of GraphSAGE using different aggregator functions. To provide a fair comparison, all models share the same implementation of their small batch iterators, loss functions, and neighborhood samplers. We select the attributed KNN graph generated by various K and S values as input, and different types of aggregators are used to train the model. The macro-f1 values of different aggregators are shown in Fig. 6. We can see that all aggregators except LSTM can achieve better performance since LSTM is not an inherent symmetric model, its inputs are processed in order, but the vectorized features of node attributes used in the fusion process have no temporal characteristics.

4.6. K sensitivity analysis

Since the number of nearest neighbors K is an important parameter of the KNN graph structure. This experiment is mainly to prove that our model is K-insensitive. Therefore, we compare the influence of the different numbers of neighbors K on the macro-f1 values of the model. It can be seen from Fig. 7(a) that the F1 score of the same sampling values and same aggregators can basically reach a higher level in the case of $K = \{5, 10, 15, 20\}$, which proves that MalDiscovery can learn useful correlation information and obtain stable results on attributed KNN graphs with different values of nearest neighbors.

4.7. Sampling values analysis

The number of neighbor sampling will also affect the learning process of the graph model. To verify this, we adopted GS-Meanpooling as the aggregation function of the model, and observe the change of macro-f1 with the sampling value at the same

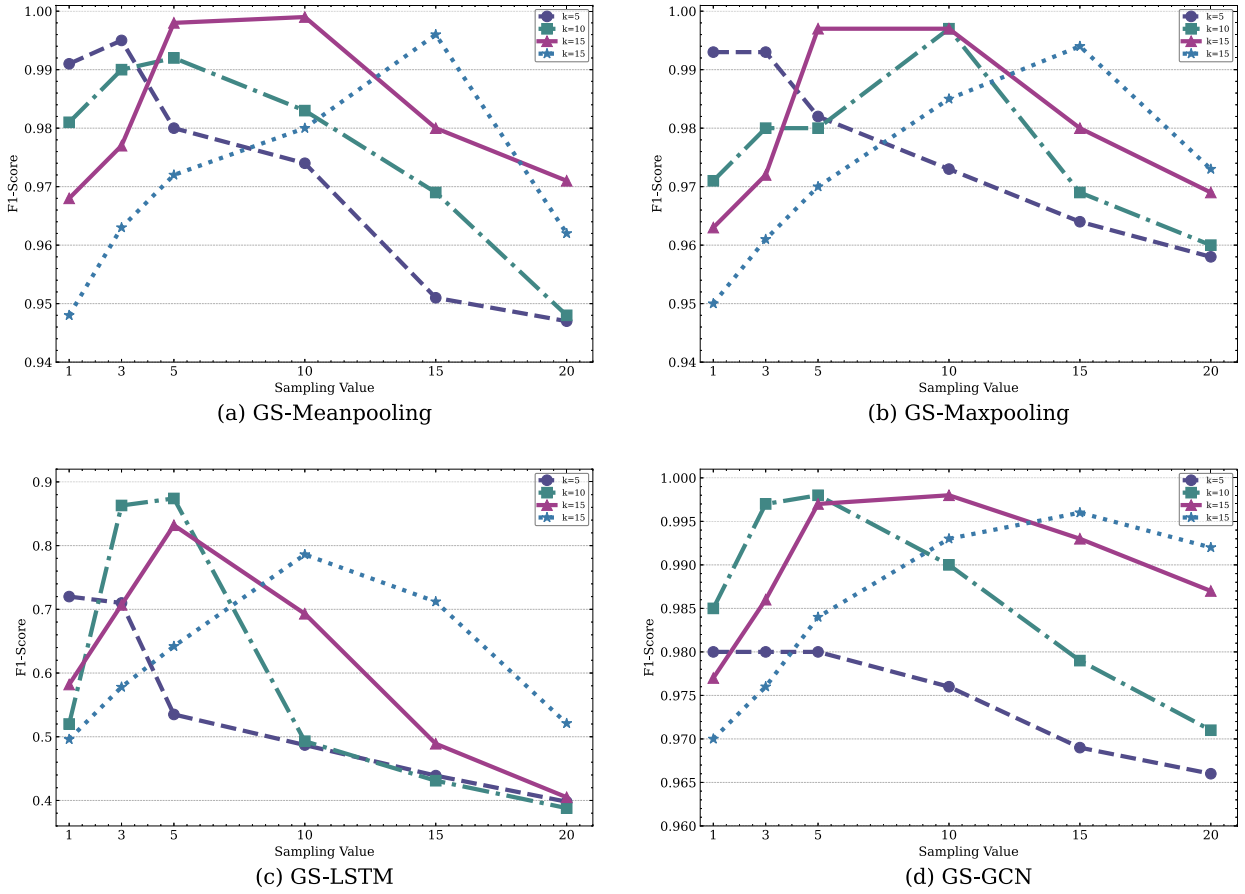


Fig. 5. Performance of MalDiscovery with Different Parameters.

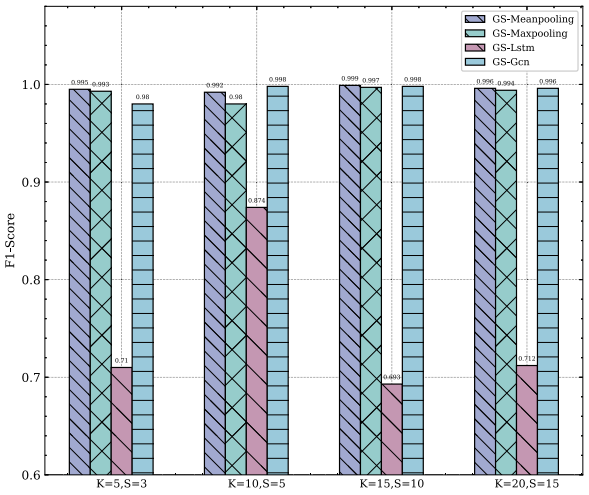
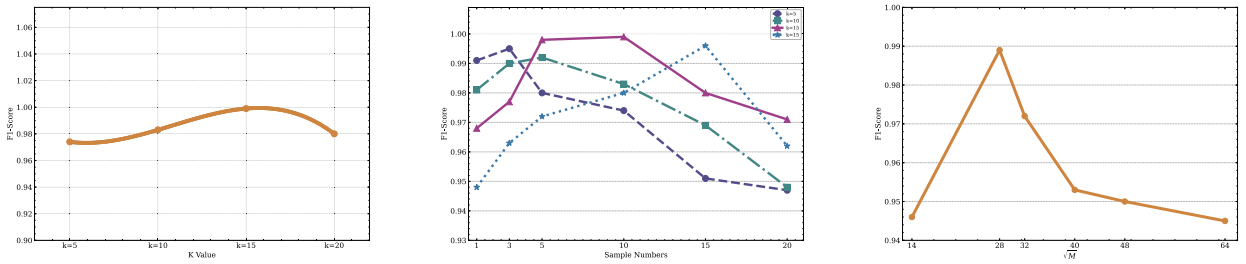


Fig. 6. F1-Score on Different Aggregators.

number of neighbors K. As can be seen from Fig. 7(b), another result is that when $S = K$ or $S = 1$, the performance will decrease significantly. It is because when $S = 1$, the KNN graph contains less correlation information, and when $S = K$, the communities in the KNN graph overlap.



(a) F1-Score on Different K Values (b) F1-Score on Different Sampling Values (c) F1-Score on Different M Values

Fig. 7. Hyperparameter Sensitivity Analysis.

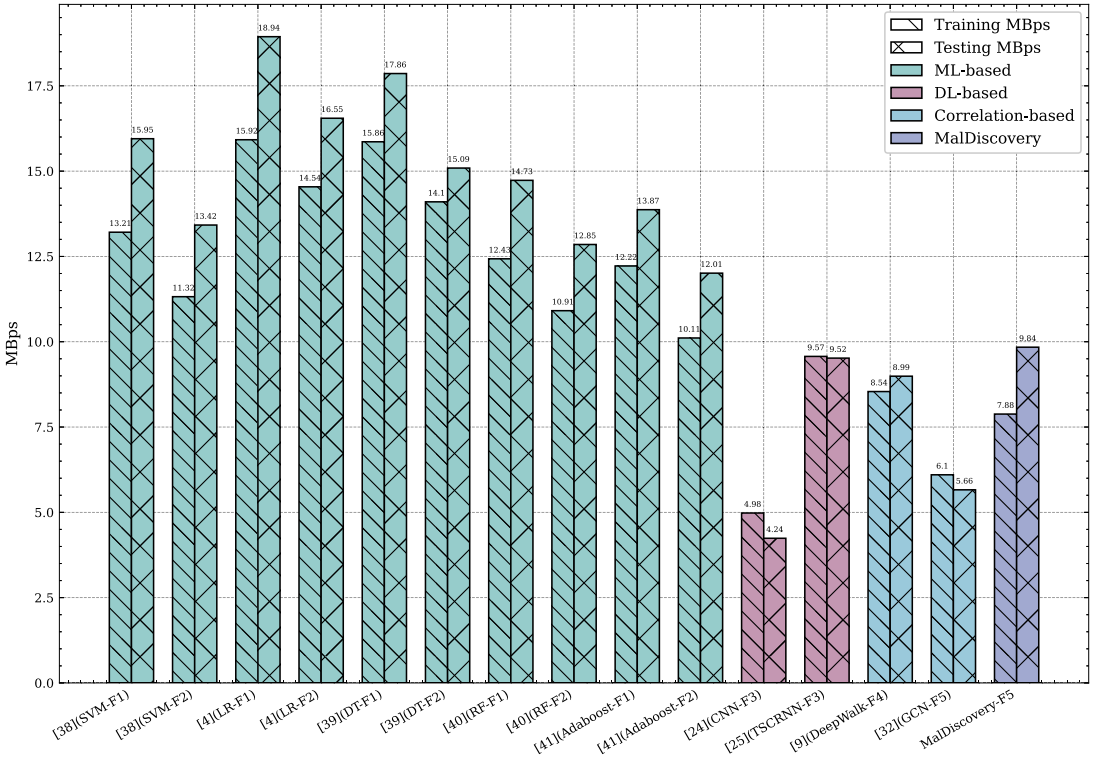


Fig. 8. Processing Efficiency of Different Methods.

4.8. *M* sensitivity analysis

We designed experiments to choose an appropriate value of M. In this paper, experiments are carried out on the CNN model to test the influence of the M value on F1-Score. The values of M are 196, 784, 1024, 1600, 2304, and 4096 bytes, respectively, that is, the generated image features are widths of 14, 28, 32, 40, 48, 64 square pictures. As shown in Fig. 7(c), different M values will lead to different classification effects of the CNN model, and the CNN effect is the best when $M = 784$. When the value of M is too small, the image features contain too little information. When the value of M is too large, the image features contain too much noise.

4.9. Processing efficiency analysis

Processing efficiency is a key factor to measure the value of a model in a real-world application. To prove that our approach has high efficiency in the testing phase and can be deployed in a real network environment, we designed an experiment to compare the average throughput of the proposed method with the existing ML-based methods, DL-based methods, and correlation-based methods during both the training and testing phases under a single epoch. All methods use the best-performed hyper-parameters. Firstly, it can be seen from Fig. 8 that although our approach may have a lower training speed compared to ML-based methods, such as SVM, LR, DT, RF, Adaboost, it still achieves acceptable testing throughput while maintaining high classification accuracy. Secondly, although DL-based methods, such as CNN and TSCRNN, have shown remarkable performance, the proposed approach

can achieve better processing efficiency, while DL-based methods are known for their high computational complexity and require powerful hardware. Thirdly, existing correlation-based methods, such as DeepWalk and GCN, may suffer from low throughput in training and testing phases since they are based on transductive models. The experimental results show that our approach utilizing the inductive GraphSAGE model can be deployed in real networks for real-time tasks of encrypted malicious traffic detection.

5. Conclusion

At present, there have been studies on botnet analysis using the graph method [39], which shows that the graph analysis method is of great significance to cyberspace security. In this work, firstly, the method constructs an attributed KNN graph by using multi-view features of encrypted traffic, including vectorized features and image features. Specifically, in the attributed KNN graph, the relationship between the nodes is constructed by the similarity of the image features of corresponding encrypted sessions, and the attributes of the nodes are derived from the vectorized features of the encrypted sessions. Secondly, to make use of the correlation of data samples in the testing phase, we propose to utilize GraphSAGE, a graph neural network composed of multiple graph layers, to capture the similarity of encrypted traffic data samples and then get more abundant embeddings forms of encrypted traffic. After learning the node embeddings, we feed it into a MLP classifier to perform binary classification on the encrypted session nodes. Since in this scenario, the malicious judgment of the encrypted session is equivalent to the binary classification of the encrypted session node in the attribute KNN graph, we transform the encrypted malicious traffic detection problem into a graph node classification problem, and thus break through the difficulty of utilizing and mining of encrypted traffic correlation. Experiment results show that our approach can effectively integrate multi-view features and improve detection efficiency.

CRediT authorship contribution statement

Yueping Hong: Data curation, Software, Writing – original draft. **Qi Li:** Conceptualization, Formal analysis. **Meng Shen:** Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This work was supported by the National Natural Science Foundation of China (62172055, U20B2045, U1836103), the National Key Research and Development Program (2019QY1404), the BUPT Project (2021XD-A09).

References

[1] Google Transparency Report, HTTPS Encryption in Chrome [EB/OL] [2022-10-01], https://transparencyreport.google.com/https/overview?hl=En, 2022.
[2] B. Anderson, S. Paul, D. McGrew, Deciphering malware's use of TLS (without decryption), J. Comput. Virol. Hacking Tech. 14 (3) (2018) 195–211.
[3] B. Anderson, D. McGrew, Identifying encrypted malware traffic with contextual flow data, in: Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, 2016, pp. 35–46.
[4] B. Anderson, D. McGrew, Machine learning for encrypted malware traffic classification: accounting for noisy labels and non-stationarity, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2017, pp. 1723–1732.
[5] MITRE ATT&CK, MITRE ATT&CK Command and Control Report [EB/OL], https://attack.mitre.org/techniques/T1571/, 2022.
[6] M. Shen, K. Ye, X. Liu, et al., Machine learning-powered encrypted network traffic analysis: a comprehensive survey, IEEE Commun. Surv. Tutor. (2022).
[7] X. Huang, Q. Song, F. Yang, et al., Large-scale heterogeneous feature embedding, Proc. AAAI Conf. Artif. Intell. 33 (01) (2019) 3878–3885.
[8] X. Huang, J. Li, X. Hu, Accelerated attributed network embedding, in: Proceedings of the 2017 SIAM International Conference on Data Mining, Society for Industrial and Applied Mathematics, 2017, pp. 633–641.
[9] Y. Ding, G. Zhu, D. Chen, et al., Adversarial sample attack and defense method for encrypted traffic data, IEEE Trans. Intell. Transp. Syst. 23 (10) (2022) 18024–18039.
[10] Z. Fu, M. Liu, Y. Qin, et al., Encrypted malware traffic detection via graph-based network analysis, in: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, 2022, pp. 495–509.
[11] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: ICLR 2017, 2017.
[12] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, 2017.
[13] G. Li, K. Ota, M. Dong, et al., DeSVig: decentralized Swift vigilance against adversarial attacks in industrial artificial intelligence systems, IEEE Trans. Ind. Inform. 16 (5) (2019) 3267–3277.
[14] G. Gs Creech, J. Hu, A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns, IEEE Trans. Comput. 63 (4) (2014) 807–819.
[15] H. Zhang, C. Papadopoulos, D. Massey, Detecting encrypted botnet traffic, in: 2013 Proceedings IEEE INFO-COM, Turin, 2013, pp. 3453–3458.
[16] Sourabh Saxena, Demystifying Malware Traffic, August 2016, pp. 1735–1780.
[17] G.S. Poh, D.M. Divakaran, H.W. Lim, et al., A survey of privacy-preserving techniques for encrypted traffic inspection over network middleboxes, arXiv preprint, arXiv:2101.04338, 2021.

[18] Z. Wang, K.W. Fok, V.L.L. Thing, Machine learning for encrypted malicious traffic detection: approaches, datasets and comparative study, Comput. Secur. 113 (2022) 102542.
[19] M. Shen, M. Wei, L. Zhu, et al., Classification of encrypted traffic with second-order Markov chains and application attribute bigrams, IEEE Trans. Inf. Forensics Secur. 12 (8) (2017) 1830–1843.
[20] R. Dai, C. Gao, B. Lang, et al., SSL malicious traffic detection based on multi-view features, in: Proceedings of the 2019 9th International Conference on Communication and Network Security, 2019, pp. 40–46.
[21] J. Liu, Y. Zeng, J. Shi, et al., MalDetect: a structure of encrypted malware traffic detection, Comput. Mater. Continua 2019 (60) (2019) 721–739.
[22] M. Shen, Y. Liu, L. Zhu, et al., Fine-grained webpage fingerprinting using only packet length information of encrypted traffic, IEEE Trans. Inf. Forensics Secur. 16 (2020) 2046–2059.
[23] C. Dong, Z. Lu, Z. Cui, et al., MBTree: detecting encryption rats communication using malicious behavior tree, IEEE Trans. Inf. Forensics Secur. 16 (2021) 3589–3603.
[24] Y. Sharon, D. Berend, Y. Liu, et al., Tantra: timing-based adversarial network traffic reshaping attack, IEEE Trans. Inf. Forensics Secur. 17 (2022) 3225–3237.
[25] H. Li, K. Ota, Dong M. Learning, IoT in edge: deep learning for the Internet of things with edge computing, IEEE Netw. 32 (1) (2018) 96–101.
[26] W. Wang, M. Zhu, J. Wang, et al., End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), IEEE, 2017, pp. 43–48.
[27] T. Shapira, Y. Shavitt, FlowPic: a generic representation for encrypted traffic classification and applications identification, IEEE Trans. Netw. Serv. Manag. 18 (2) (2021) 1218–1232.
[28] K. Lin, X. Xu, H. Gao, TSCRNN: a novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT, Comput. Netw. 190 (2021) 107974.
[29] W. Bazuhair, Lee W. Detecting, Malign encrypted network traffic using perlin noise and convolutional neural network, in: 2020 10th Annual Computing and Communication Workshop and Conference (CCWC), 2020.
[30] Wang Pan, Chen Xuejiao, SAE-based encrypted traffic identification method, Comput. Eng. 44 (11) (2018) 140.
[31] G. Shao, X. Chen, X. Zeng, et al., Deep learning hierarchical representation from heterogeneous flow-level communication data, IEEE Trans. Inf. Forensics Secur. 15 (2019) 1525–1540.
[32] M. Shen, J. Zhang, L. Zhu, et al., Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 2367–2380.
[33] W. Wang, Y. Shang, Y. He, et al., BotMark: automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors, Inf. Sci. 511 (2020) 284–296.
[34] T. Cui, G. Gou, G. Xiong, et al., SiamHAN: IPv6 address correlation attacks on TLS encrypted traffic via Siamese heterogeneous graph attention network, in: USENIX Security Symposium, 2021, pp. 4329–4346.
[35] T.D. Pham, T.L. Ho, T. Truong-Huu, et al., Mappgraph: mobile-app classification on encrypted network traffic using deep graph convolution neural networks, in: Annual Computer Security Applications Conference, 2021, pp. 1025–1038.
[36] M. Shen, Y. Liu, L. Zhu, et al., Optimizing feature selection for efficient encrypted traffic classification: a systematic approach, IEEE Netw. 34 (4) (2020) 20–27.
[37] https://www.zeek.org/.
[38] https://www.stratosphereips.org/datasets-ctu13.
[39] https://www.virustotal.com/gui/.
[40] https://mcfp.felk.cvut.cz/publicDatasets/.
[41] A.S. Shekhawat, F. Di Troia, M. Stamp, Feature analysis of encrypted malicious traffic, Expert Syst. Appl. 125 (2019) 130–141.
[42] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, et al., Characterization of encrypted and vpn traffic using time-related, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP), 2016, pp. 407–414.
[43] K.S.A.N. Zincir-Heywood, How far can we push flow analysis to identify encrypted anonymity network traffic?
[44] R. Alshammari, A.N. Zincir-Heywood, Can encrypted traffic be identified without port numbers, IP addresses and payload inspection?, Comput. Netw. 55 (6) (2011) 1326–1350.